THOMPSON RIVERS UNIVERSITY

Software Design Patterns (Fall 2024) - SENG 4130

Automated Respiratory Monitoring System (BreathWatch)

Luka Aitken (T00663672)

Toma Aitken (T00663680)

December 6, 2024

Table of Contents

List of Figures

List of Tables

1 Introduction

Around the world, about 80 percent of people suffer from breathing dysfunction [1]. People have different types of breathing patterns, which some could be abnormal. These abnormal breathing patterns could be caused by anxiety, anemia, lung conditions or even heart conditions. With these conditions, it can lead to some dangerous situations like someone having heart failure or other issues [2]. When someone is in this type of situation, the only place that has the equipment to help with the situations as well as to see your breathing patterns is the hospital. These systems are not very portable and can only be accessed at the hospital which can cause some issues when arriving. At the hospital, it could either be very quick to check if your condition is in a dangerous state or it could take a very long time to see your breathing patterns if your condition is less serious. Also, it could cost lots of money to even use the system which some people might not even have enough money to afford the check there breathing. Additionally, these systems at households cost lots of money to even afford one, making the situation worse if a problem arises. With all these points and issues that could arise, using an automated respiratory monitoring system is the key to monitoring these issues quicker, portable, and cheaper and that's where our system comes in.

To begin with our Automated Respiratory Monitoring System, we will come up with different possible solutions that will require minimal effort and ease of use for the users, while following the project requirements and constraints. The goal was to come up with a solution to a portable remote health monitoring system program using Java for the functionality and Java Swing for the user-friendly view, which will allow users to monitor their breathing and make sure that it is kept in check at an affordable price. After choosing the final design for our system, we will create a prototype version of the program, where we will do the necessary tests to ensure that they fit with the project requirements and constraints.

Throughout this report, we will discuss the problems we faced with the designing and building the Automated Respiratory Monitoring System. We will also discuss other solutions that weren't used and how they inspired the final solution. Additionally, the environmental, societal, safety and economic considerations are other key factors that will be discussed throughout this report.

2 Design Problem

2.1 Problem Definition

Access to effective respiratory monitoring remains a significant challenge for individuals, particularly those with chronic respiratory conditions. Traditional monitoring systems are often limited to clinical settings, making real-time assessment difficult for patients outside of hospital environments. This lack of accessibility can lead to delayed recognition of respiratory issues, increasing the risk of severe health complications [3]. Furthermore, the high costs associated with existing monitoring solutions often prevent individuals, especially in low-income communities, from obtaining the necessary care [4].

To address these challenges, our project aims to develop a software-based Automated Remote Respiratory Monitoring System that simulates real-time data collection and monitoring of respiratory patterns. While the system will not involve physical devices, it will utilize generated data to demonstrate how users can track their breathing rates and receive alerts for significant changes. By providing a user-friendly interface that aggregates and displays respiratory data, the software will empower users to manage their respiratory health proactively. This innovative approach not only enhances understanding of respiratory monitoring but also demonstrates the potential for accessible, technology-driven health solutions.

2.2 Design Requirements

This section outlines the functions, objectives, constraints, and requirements of the BreathWatch monitoring system, detailing the system's intended features, performance standards, and constraints.

2.2.1 Functions

- F-1: The system should enable users to monitor their breathing patterns and detect any anomalies or discrepancies.
- F-2: The system must identify and report significant events such as extended pauses in breathing.
- F-3: All user information should remain private, with access restricted to authorized users only.

Performance	8
Safety	9
Simplicity	7
Maintenance	7
Affordability	8

Table 2.2.1 - Importance Chart.

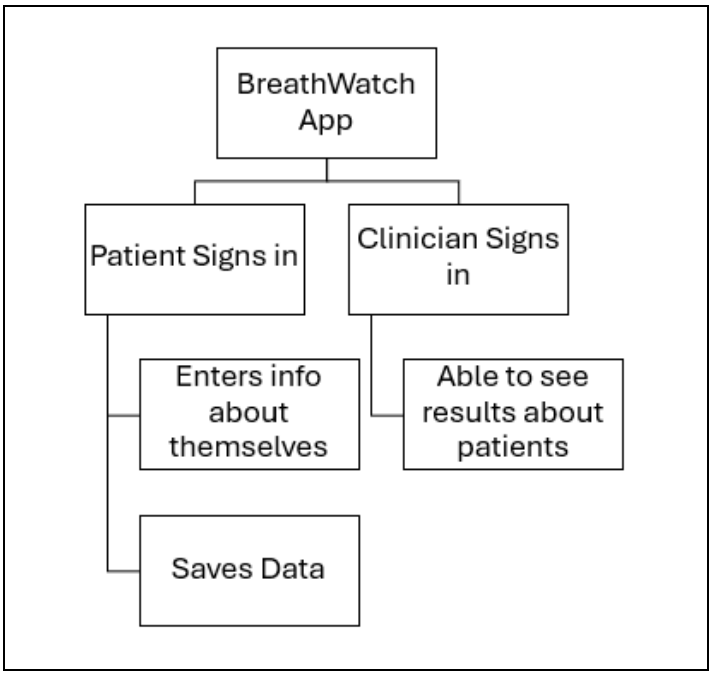


Figure 2.1 - Function Tree

2.2.2 Objectives

- O-1: The system will manage both dynamic and static data (e.g., breathing patterns and heart rate)
- O-2: The design will incorporate at least three design patterns, such as Observer, Singleton, and Strategy, to ensure modularity and maintainability.
- O-3: The user interface will be developed using Java Swing, emphasizing user-friendliness and intuitiveness.
- O-4: The system will be scalable to support future updates, including potential integration with physical sensors.

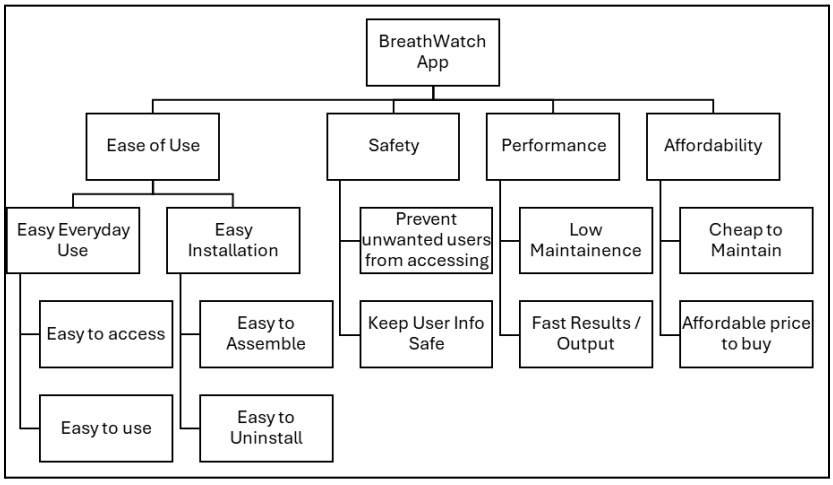


Figure 2.2 - Objective Tree

2.2.3 Constraints

- C-1: Development must be in Java, utilizing the Java Swing framework.
- C-2: The solution must incorporate at least three design patterns to demonstrate modularity and maintainability.
- C-3: The system should maintain robust performance even as the number of users and the volume of data increases.
- C-4: Private patient information must only be accessible to authorized clinicians.

2.2.4 Functional Requirements

- FR-1: The user should be able to open the program without issues.
- FR-2: The system should allow users to sign in as either a Patient or a Clinician.
- FR-3: Patients should have the ability to enter their personal and health-related information.
- FR-4: Clinicians should be able to view and analyze patient data collected by the system.

2.2.5 Non-Functional Requirements

- NF-1: Only registered users with valid credentials should be able to access the system.
- NF-2: Clinicians should have full access to patient data, while other users have restricted access.
- NF-3: Results and sensitive information are visible only to clinicians.

2.2.6 Scope & Complexity

- SC-1: The system will be compatible with Windows 11 as its primary operating environment.
- SC-2: It will be accessible in environments like pharmacies and hospitals, enhancing healthcare availability.
- SC-3: Patient data is securely displayed only to users with clinician-level access.

3 Solution

3.1 Solution 1

Our initial solution involved building the system using the Singleton, Observer, and Decorator design patterns. The Singleton pattern was intended to handle the login functionality, ensuring that users could securely enter their credentials. It would validate the credentials and grant access to the system if they were correct, directing the user to the appropriate main page based on their role (patient or clinician). The Decorator pattern was proposed to differentiate between user types by dynamically layering components. For instance, clinicians would have access to additional features not available to patients, such as a more detailed data overview. Meanwhile, the Observer pattern would facilitate real-time monitoring by tracking patient breathing data from a simulated dataset. This data would then be analyzed to detect irregularities such as long pauses or abnormal patterns, providing feedback such as "healthy breathing" or highlighting potential issues.

While conceptually promising, this solution presented significant challenges in implementation due to the interactions between the chosen design patterns. The primary issue arose from using the Singleton pattern for login. By definition, the Singleton pattern restricts instantiation to a single instance, which conflicted with the system's need for dynamic multi-user access. This limitation meant that the login process could only accommodate a single set of hard-coded credentials, making it unsuitable for an application that required independent access for multiple users with differentiated permissions.

Additionally, the constraints imposed by the Singleton pattern undermined the effectiveness of the Decorator pattern. Although the Decorator was intended to provide user-specific views, the inability to manage multiple distinct users prevented the system from leveraging the full potential of this pattern. The combination of these issues resulted in a system that lacked the flexibility and scalability required to support both patients and clinicians effectively. Consequently, we determined that this solution was incompatible with the project’s requirements and opted to explore alternative approaches that better aligned with our goals.

3.2 Final Solution

Our final solution leverages the Singleton, Iterator, Template, and Observer design patterns to create a comprehensive, user-friendly system for respiratory monitoring. It addresses the limitations of Solution 1 by offering dynamic multi-user support, role-specific interfaces, and robust real-time data monitoring. These design improvements make this approach scalable, maintainable, and aligned with the project's goals.

The Singleton pattern is now confined to managing shared resources such as a secure DataConnection. This ensures centralized control over database operations while avoiding the pitfalls encountered in Solution 1. The Iterator pattern is introduced to provide controlled traversal of vital signs data, such as breathing and heart rates, allowing clinicians to navigate patient records efficiently without exposing the internal structure of the data. The Template pattern defines a flexible framework for constructing user interfaces, with the HealthFrame class serving as a base for role-specific subclasses, such as PatientFrame and ClinicianFrame. This separation of concerns enables each user type to access features tailored to their needs. The Observer pattern enhances real-time monitoring, triggering notifications and alerts whenever irregular vital signs are detected, ensuring timely responses to potential health issues. This final solution's advantages are summarized in the following table:

Feature	Solution 1	Final Solution
Multi-user Access	Limited due to Singleton restrictions.	Fully supports dynamic login for multiple users.
User-specific Interfaces	Partially supported through the Decorator pattern but limited by Singleton.	Fully implemented using Iterator and Template patterns.
Real-time Data Monitoring	Basic functionality with Observer, but lacked robustness.	Enhanced with Observer for real-time tracking and analysis.
Scalability	Limited by pattern conflicts.	Highly scalable due to modular design.
Ease of Maintenance	Complex due to incompatible pattern interactions.	Cleanly implemented with clear separation of concerns.
Data Privacy and Security	Basic privacy measures.	Secure login and session management ensure data integrity.

Table 3.2.1- Comparison Table of Solutions

In this solution, the Observer pattern plays a critical role in real-time monitoring, ensuring alerts are generated for irregular breathing or heart rates. The Iterator pattern enhances data management by enabling controlled navigation of lists such as breathingRates and heartRates within the VitalSignsData class. The Singleton pattern ensures a single instance of DataConnection is used, optimizing resource usage and maintaining secure data access. Finally, the Template pattern provides a consistent and extensible structure for creating frames, ensuring that subclasses like PatientFrame and ClinicianFrame can be customized without duplicating code. Together, these patterns create a robust, scalable, and user-focused solution, making it the optimal choice for this project.

		Solutions			
		Solution 1		Final Solution	
Criteria	**Weight**	**Score**	**Partial Score**	**Score**	**Partial Score**
Performance	0.40	8/10	0.320	8/10	0.320
Usability	0.20	3/5	0.120	4/5	0.160
Scalability	0.25	7/15	0.116	12/15	0.200
Reliability	0.15	6/10	0.090	8/10	0.120
Sum	1.00		0.646		0.8

Table 3.2.2: Decision matrix

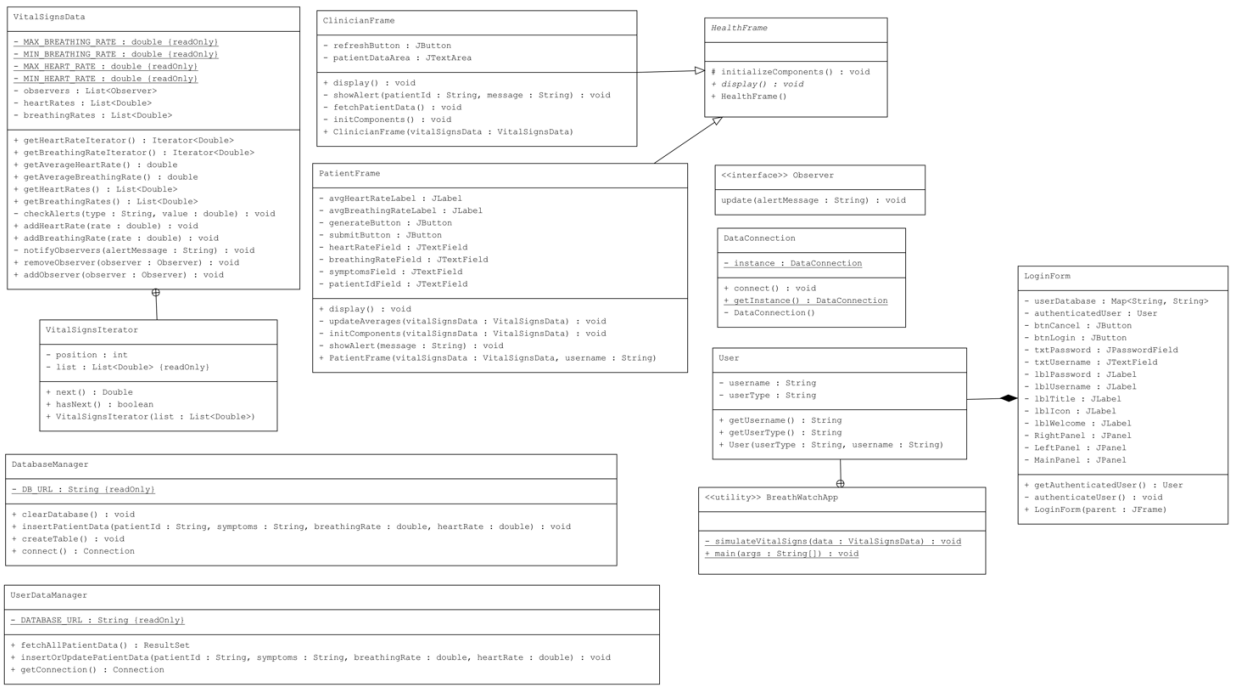


Figure 3.2.1 - Full UML Diagram

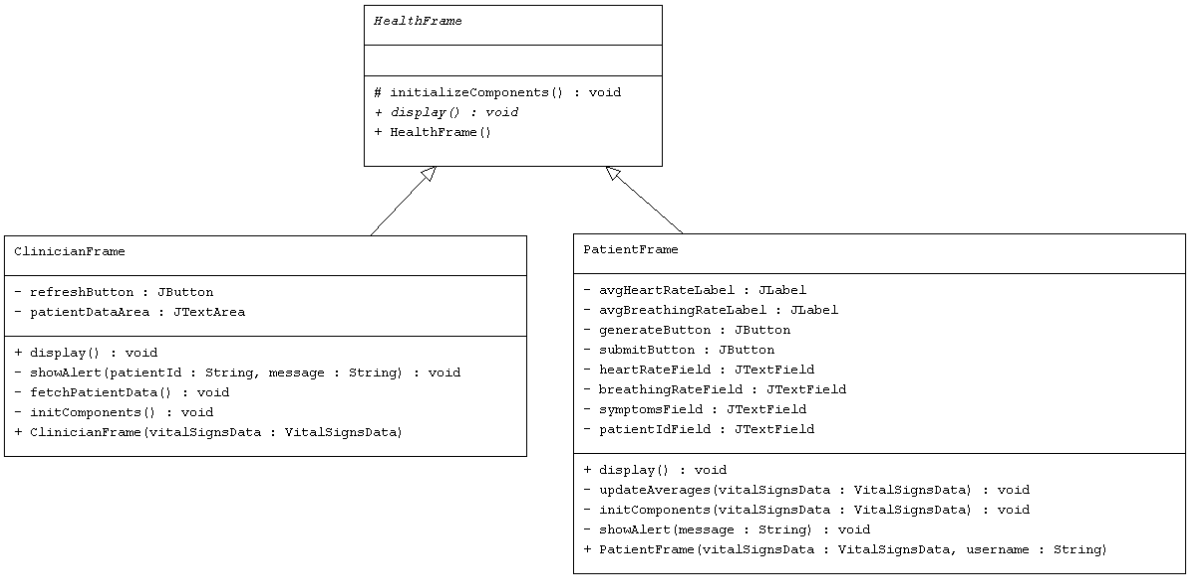


Figure 3.2.2 - UML Diagram of Template Pattern

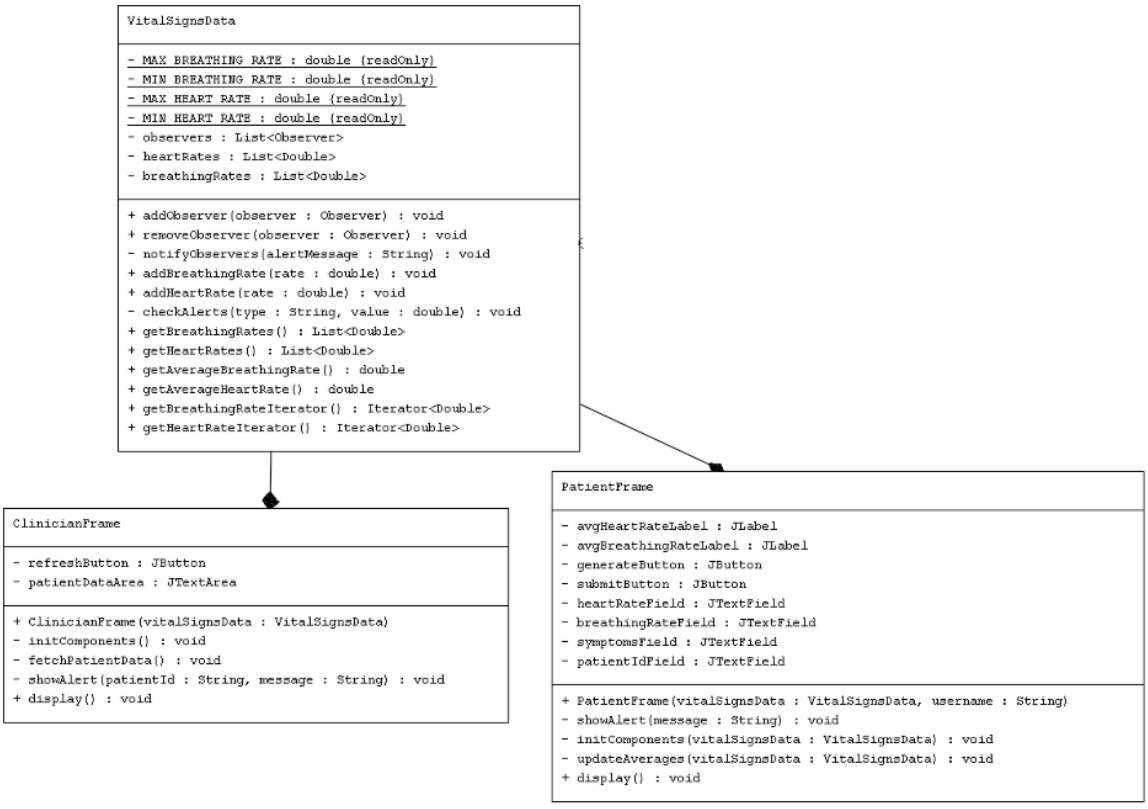


Figure 3.2.3 - UML Diagram of Iterator Pattern

List of All Files in BreathWatch Application	
File Name	Description
BreathWatchApp.java	This is the main entry point for the BreathWatchApp application, handling user authentication through the LoginForm and determining whether to display the PatientFrame or ClinicianFrame based on the authenticated user's role. It also simulates vital signs data, such as heart rate and breathing rate, and establishes a data connection.
DatabaseManager.java	This class manages the SQLite database connection for the BreathWatchApp. It includes methods to connect to the database, create a PatientData table (if it doesn't exist), insert patient data, and clear all patient records from the database
DataConnection.java	This is a singleton class that ensures only one instance of the DataConnection object exists throughout the application. It provides a method to simulate connecting to a data source.
HealthFrame.java	This is an abstract class extending JFrame that serves as a base for creating patient and clinician-specific frames. It sets up common properties for the application window, such as the title, icon, and layout, and defines an abstract display method for subclasses to implement their specific UI content.
PatientFrame.java	This class extends HealthFrame and represents the patient interface of the BreathWatch app. It includes components such as text fields for entering patient ID, symptoms, breathing rate, and heart rate, along with buttons for submitting data and generating random values. The submitButton saves the data to the database, and checks for abnormal breathing or heart rate values,

	displaying an alert if necessary. The generateButton populates the fields with random values and updates the average rates displayed on the frame. The initComponents method sets up the layout and the frame's components, while updateAverages recalculates and updates the average breathing and heart rates.
ClinicianFrame.java	This class extends HealthFrame and provides a user interface for clinicians to view and refresh patient data, including vital signs such as breathing and heart rate, fetched from a database. It uses a SwingWorker to asynchronously load and display patient data while checking for abnormal values and alerting the clinician if necessary.
LoginForm.java	This class creates a login dialog for the BreathWatch application, where users can authenticate as either a patient or a clinician. It uses a simple in-memory user database (username-password pairs) for validation and provides feedback for failed login attempts. Upon successful login, the authenticated user's details are stored and the dialog is closed.
Observer.java	This interface defines a method (update) that allows classes to receive updates (in the form of alert messages) when they are notified by an observable subject. It follows the Observer design pattern, enabling loose coupling between components.
UserDataManager.java	This class manages interactions with a SQLite database for patient data, including methods to insert or update patient information (such as symptoms, breathing rate, and heart rate) and fetch all patient data ordered by timestamp. It provides database connection handling and executes SQL queries to update or retrieve patient records.

VitalSignsData.java	This class stores and manages patient vital signs data, including breathing rates and heart rates, while providing alerting functionality for critical values (e.g., heart rate outside 40-120 bpm range). It also implements the Observer pattern to notify registered observers (like patients and clinicians) of critical conditions and calculates average vital signs. Iterators are provided to traverse through the list of data points.
sample.db	This database file stores SQL queries generated from the patient view, allowing clinicians to access and review the data at a later time.

Table 3.2.3 - List of All Files in BreathWatch Application

3.2.1 Features

The final solution includes a robust set of features designed to provide an efficient and user-friendly respiratory monitoring system. Each feature is supported by specific methods or design patterns, ensuring reliability, scalability, and security.

3.2.1.1 Secure Login System

The login system is a critical component of the solution, ensuring only authorized users can access the application. By leveraging the Singleton pattern, the system manages a single instance of a secure database connection. This not only centralizes control over user authentication but also reduces resource overhead. The LoginForm.validateCredentials() method validates the username and password entered by the user. If credentials are valid, users are granted access to role-specific interfaces tailored to either patients or clinicians, enhancing privacy and data security. With our system, we have 3 patient accounts and 1 clinician account each with their own passwords. Each account has a dedicated id based on their username.

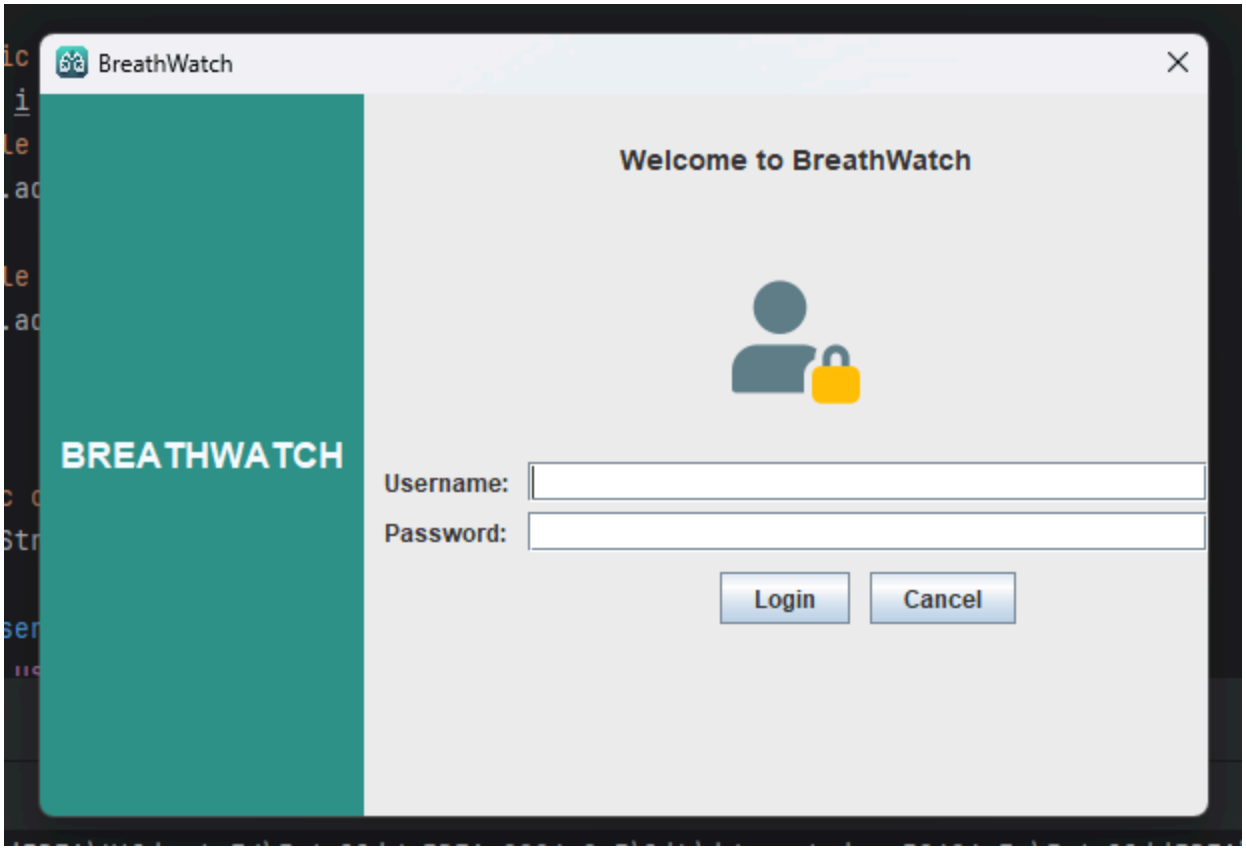


Figure 3.2.4 - Login Page for BreathWatch

3.2.1.2 Patient Monitoring Interface

Patients interact with a streamlined interface that allows them to input details and initiate a monitoring session. During these sessions, the system tracks vital signs, including breathing and heart rates, in real time. The PatientFrame.startMonitoring() method handles the initialization of the monitoring session, while the VitalSignsData.addRecord() method updates the breathing and heart rate data dynamically. This feature empowers patients to actively monitor their health and detect any potential issues early. In the figure 3.2.5 showing the system, all data is generated in the background as shown in figure 3.2.6.

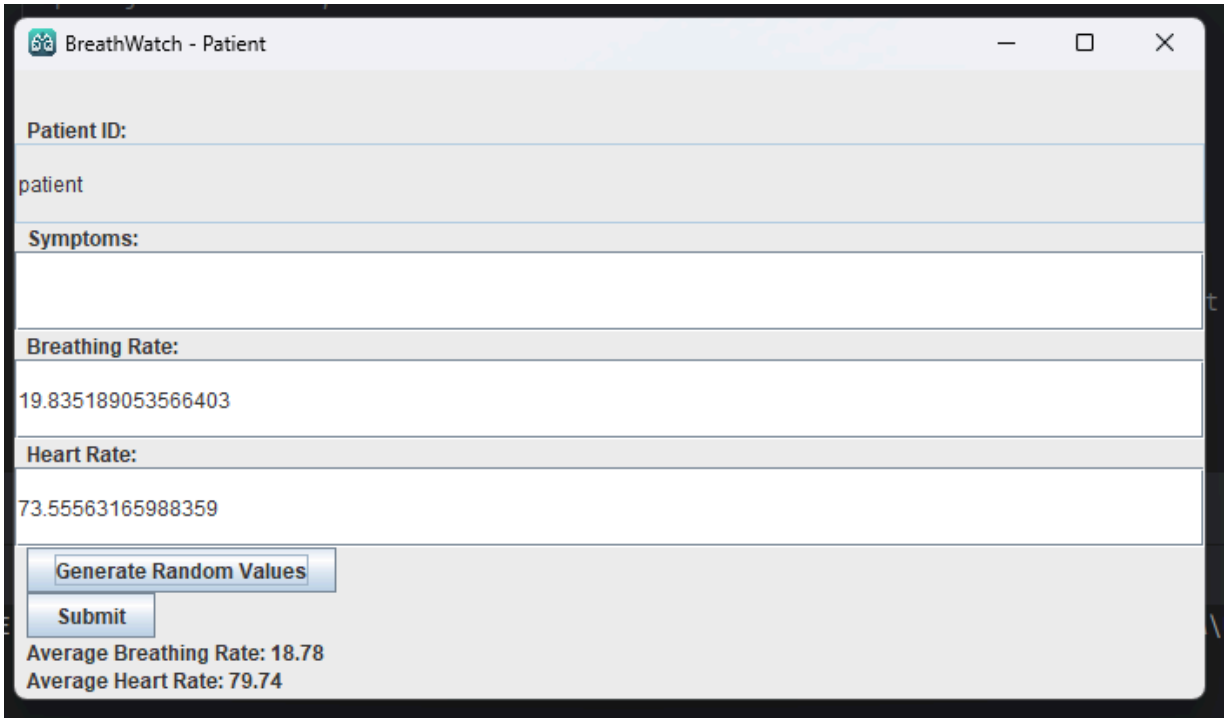


Figure 3.2.5 - Patient View

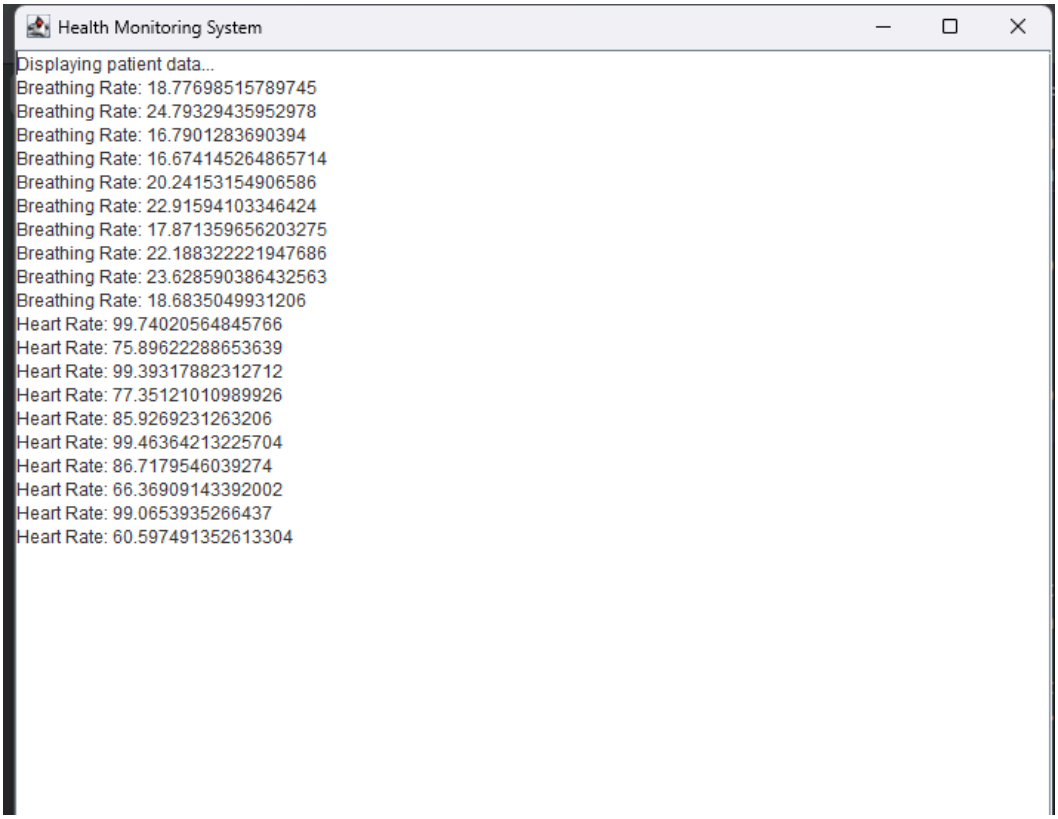


Figure 3.2.6 - Backend Data Generating

3.2.1.3 Database and Data Storage

The system uses a relational database to store essential user and health data, including user ID, patient ID, symptoms, breathing rate, heart rate, and timestamp. This data is structured across multiple tables, with the "Users" table storing user information and the "Vital Signs" table tracking real-time health data for each patient. Each entry in the "Vital Signs" table includes a timestamp to ensure accurate tracking of health metrics over time. To ensure data security, user credentials are hashed, and all interactions with the database are managed through a Singleton DataConnection class, which maintains a single, secure connection. Methods such as addPatientData() and getPatientData() handle data insertion and retrieval, allowing the system to manage patient data efficiently while ensuring both security and performance.

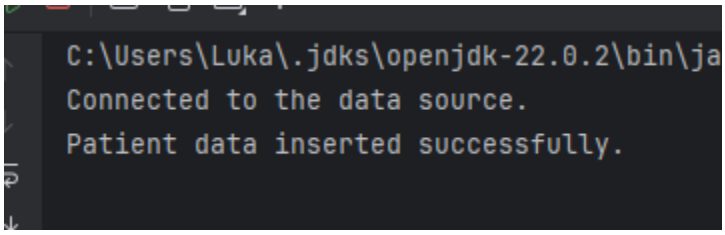
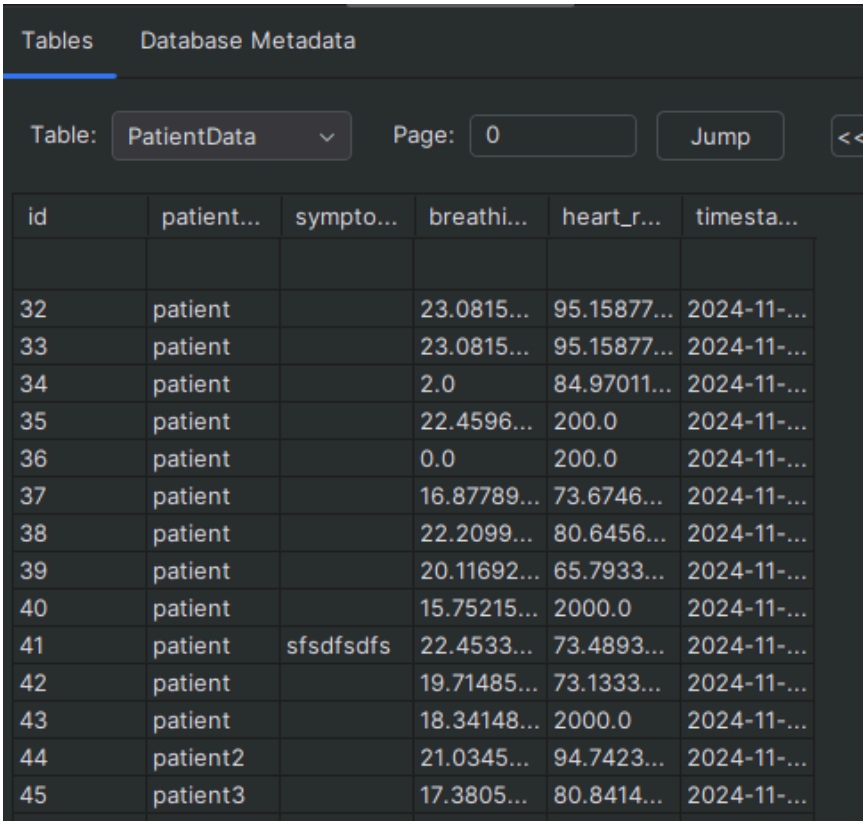


Figure 3.2.7 - Data Inserted Message



id	patient...	sympto...	breathi...	heart_r...	timesta...
32	patient		23.0815...	95.15877...	2024-11-...
33	patient		23.0815...	95.15877...	2024-11-...
34	patient		2.0	84.97011...	2024-11-...
35	patient		22.4596...	200.0	2024-11-...
36	patient		0.0	200.0	2024-11-...
37	patient		16.87789...	73.6746...	2024-11-...
38	patient		22.2099...	80.6456...	2024-11-...
39	patient		20.11692...	65.7933...	2024-11-...
40	patient		15.75215...	2000.0	2024-11-...
41	patient	sfsdfsdfs	22.4533...	73.4893...	2024-11-...
42	patient		19.71485...	73.1333...	2024-11-...
43	patient		18.34148...	2000.0	2024-11-...
44	patient2		21.0345...	94.7423...	2024-11-...
45	patient3		17.3805...	80.8414...	2024-11-...

Figure 3.2.8 - SQL Database

3.2.1.4 Alert Notifications

The system employs the Observer pattern to provide real-time alerts whenever irregularities in breathing or heart rates are detected. Alerts are generated through the Observer.notifyObservers() method, which triggers notifications for the user. Additionally, the VitalSignsData.analyzeData() method continuously monitors for anomalies in the recorded data, ensuring users are promptly informed of any abnormalities. This feature acts as a proactive health management tool, encouraging users to seek medical attention when necessary.

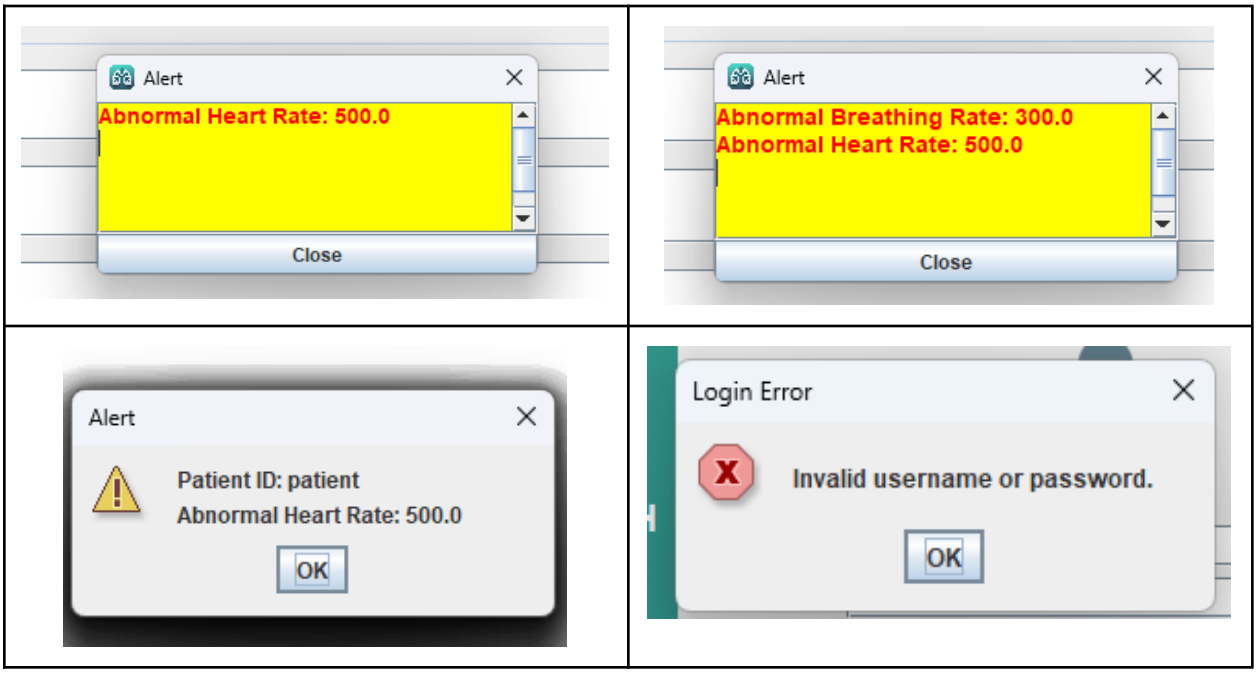


Table 3.2.4 - Images of Alert Types

3.2.1.5 Clinician Dashboard

The clinician dashboard provides a centralized interface for viewing and analyzing patient data. Clinicians can access detailed respiratory and heart rate records for all registered patients. The Iterator pattern enhances this feature, allowing clinicians to traverse patient records efficiently using the Iterator.next() method. The ClinicianFrame.displayPatientList() method organizes patient data in an intuitive format, enabling clinicians to identify trends or concerns quickly and provide timely recommendations.

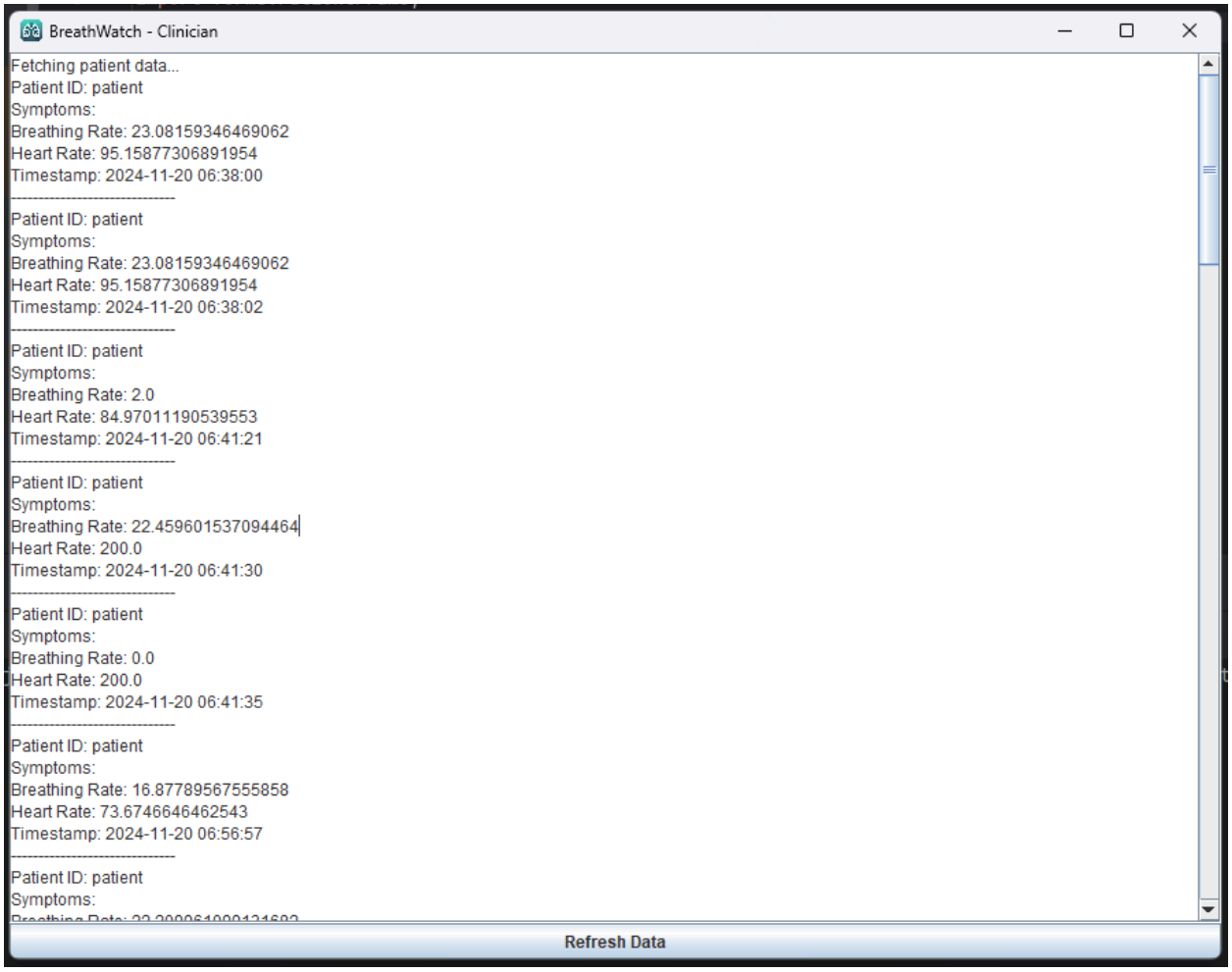


Figure 3.2.9 - Clinician Page

3.2.1.6 User-Specific Interfaces

The system employs the Template pattern to deliver customized interfaces for patients and clinicians. The HealthFrame.createFrame() abstract method serves as the foundation for creating tailored interfaces in the PatientFrame and ClinicianFrame subclasses. This modular approach ensures that patients and clinicians only see features relevant to their role, reducing complexity and improving usability. Additionally, this design allows for easy future expansions, such as adding new user types or features, without requiring significant code modifications.

The features outlined above showcase the final solution's ability to meet the diverse needs of patients and clinicians. From secure authentication to real-time monitoring and tailored user interfaces, the system provides a comprehensive approach to respiratory health management. By integrating advanced design patterns and intuitive workflows, the solution ensures scalability, maintainability, and a user-focused experience.

3.2.2 Environmental, Societal, Safety, and Economic Considerations

When designing the BreathWatch system, we prioritized environmental, societal, safety, and economic considerations to ensure that the system is both effective and responsible in its impact. Below, we detail how these considerations were integrated into our design.

3.2.2.1 Environmental Considerations

In the development of the BreathWatch system, we aimed to minimize its environmental footprint. The system was designed to be lightweight and efficient, reducing the computational resources required during use. This optimization helps lower energy consumption, indirectly contributing to lower carbon emissions. By focusing on efficient algorithms and reducing unnecessary computations, the system ensures that resources are used effectively without causing excessive environmental impact. Furthermore, the system's simplicity and minimal hardware requirements ensure that it can run on devices with lower energy demands, which is particularly important when considering the overall environmental impact of digital solutions. Unlike cloud-based systems, which rely on extensive server infrastructure, BreathWatch uses SQLite for local data storage. This decision reduces the need for large-scale cloud servers and their associated energy demands. With SQLite, patient data is stored locally, reducing the environmental costs associated with data transmission and cloud storage infrastructure. This approach also offers the benefit of ensuring patient data privacy by avoiding reliance on external data centers [5].

3.2.2.2 Societal Considerations

BreathWatch was designed with accessibility in mind to ensure it is usable by individuals with varying levels of technical expertise. We aimed to create an intuitive and user-friendly interface so that both patients and clinicians could use the system without facing a steep learning curve. The simplicity of the design ensures that even individuals with minimal technical knowledge can easily input their details and monitor their vital signs, improving health accessibility for all users. The system also contributes positively to society by enabling early detection of irregularities in patient health data, encouraging proactive healthcare. With real-time monitoring and alert notifications, patients are made aware of potential issues, prompting them to seek medical attention before conditions worsen. This feature enhances preventative healthcare and potentially improves overall health outcomes, reducing the burden on healthcare systems.

3.2.2.3 Safety Considerations

The safety of users was a primary concern during the design process. The BreathWatch system uses secure login protocols to ensure that only verified users can access sensitive patient data. This restricts unauthorized access and ensures privacy for both patients and clinicians. Furthermore, the system only allows clinicians to view patient data, maintaining the confidentiality of health information. The system's real-time alert feature also plays an essential role in safety, as it notifies both patients and clinicians when irregular vital signs are detected. This ensures timely intervention if necessary, reducing the risk of health complications going

unnoticed. The system was thoroughly tested to ensure its reliability and safety, ensuring that the data provided is accurate and trustworthy.

3.2.2.4 Economic Considerations

From an economic perspective, BreathWatch was designed to be cost-effective both for development and for end-users. By focusing on optimizing the codebase and reducing the complexity of the system, we have ensured that the software can be deployed on lower-end devices without compromising performance. This decision minimizes the costs associated with hardware upgrades for end users, making the system accessible to a broader range of individuals, including those in lower-income regions. Additionally, by using SQLite for local storage, we eliminate the need for expensive cloud storage solutions, reducing operational costs. This approach makes the system more affordable to deploy and maintain, especially for small healthcare providers or clinics with limited budgets. SQLite is lightweight, free to use, and does not require significant server infrastructure, further reducing the overall cost of ownership.

In summary, the BreathWatch system was designed with a focus on minimizing its environmental impact, improving societal health outcomes, ensuring the safety of user data, and keeping the system cost-effective. By integrating these considerations into the design, we have created a solution that not only serves the healthcare industry effectively but also promotes sustainability, accessibility, and affordability for a broad range of users.

3.2.3 Limitations

With prototyping and designing the BreathWatch system, there are bound to be limitations. For our program, we had a few limitations that affected the final results of the BreathWatch system. The main limitation that we faced was that whenever a clinician opens the system, they would get bombarded with notifications of recent patient data, which can make it somewhat annoying for the clinician. This notification feature is very important as it will notify the clinicians of any new results that are being saved. If this feature was not implemented, the clinicians could miss the new results which could cause lots of problems. Another limitation that occurred when designing BreathWatch was that we were unable to implement any charts or graphs for statistics due to time constraints. These charts and graphs required us to get the libraries used to integrate with the charts and graphs which did not work. If it did work, we could show the breathing or heath rate for better visuals for the user.

3.2.4 Life-long Learning

When developing our health monitoring system, there were some things that we needed to learn in order to have the patient data stored safely and easily accessed by clinicians when needed. With BreathWatch, we chose to use a SQL database which all the patient data can be stored into. This database holds all the recorded submissions and results from all patients in which it can be accessed by the clinicians. Also, once a clinician opens the system, they will be alerted by new submissions. Another very important and critical part of the development process involved studying breathing patterns to differentiate between normal and abnormal readings, enabling the system to provide actionable insights. For instance, if a patient’s

breathing rate exceeds 25 breaths per minute, the system triggers a pop-up alert to flag the abnormal reading. This ensures clinicians are immediately informed and can review the data in detail within the platform, supporting timely and accurate interventions.

4 Team Work

4.1 Meeting 1

Time: September 26, 2024, 11:40 am to 12:20 pm

Agenda: Discuss possible project ideas.

Team Member	Previous Task	Completion State	Next Task
Luka Aitken	N/A	N/A	Research / Pattern Design
Toma Aitken	N/A	N/A	Research / Pattern Design

1. Talked about different ideas for a project.
2. Leaned more into the respiratory measuring devices.
3. Discussed how a gui would work, whats dynamic and statics
    a. breathing dynamic, age/gender/weight static?
4. Will be working on this this weekend to have deliverables 1 completed.

4.2 Meeting 2

Time: Oct. 6, 2024, 5:00 pm to 6:00 pm

Agenda: Writing out Intro, Problem Definition and Start Code implementing

Team Member	Previous Task	Completion State	Next Task
Luka Aitken	Research / Pattern Design	100%	Problem Def, Code building
Toma Aitken	Research / Pattern Design	100%	Intro, Project Requirements

1. Started to write out the intro and problem definition, as well as started to build the code for BreathWatch.

4.3 Meeting 3

Time: Oct. 22, 2024, 5:00 pm to 6:00 pm

Agenda: Refining code with different patterns and adjusting our first solution

Team Member	Previous Task	Completion State	Next Task
Luka Aitken	Problem Def, Code building	100%	Refining Code, Trying different patterns
Toma Aitken	Research / Pattern Design	100%	Solution 1

1. Modifying code and testing other patterns to see if they work better.
2. Write out initial solution

4.4 Meeting 4

Time: Nov 7, 2024, 7:00 pm to 10:00 pm

Agenda: Refining code and making sure that the final solution can save data.

Team Member	Previous Task	Completion State	Next Task
Luka Aitken	Refining Code, Trying different patterns	50%	Final Solution
Toma Aitken	Solution 1	100%	Considerations/Limitation

1. Refining Code and making sure that each pattern works properly and can access database.
2. Writing out considerations and limitations.

4.5 Meeting 5

Time: Nov 26, 2024, 8:00 pm to 10:00 pm

Agenda: Making PowerPoint and Refining Report.

Team Member	Previous Task	Completion State	Next Task
Luka Aitken	Refining Code, Trying different patterns	100%	PowerPoint/Refine Report
Toma Aitken	Considerations/Limitation	100%	PowerPoint/Refine Report

1. Finish Code and Parts of the report.
2. Make a powerpoint to presentate BreathWatch.

4.6 Meeting 6

Time: Dec 6, 2024, 2:00 pm to 10:00 pm

Agenda: Making PowerPoint and Refining Report.

Team Member	Previous Task	Completion State	Next Task
Luka Aitken	PowerPoint/Refine Report	100%	N/A
Toma Aitken	PowerPoint/Refine Report	100%	N/A

1. Refine the report with all info about BreathWatch and make sure that everything is good to hand in.

5 Project Management

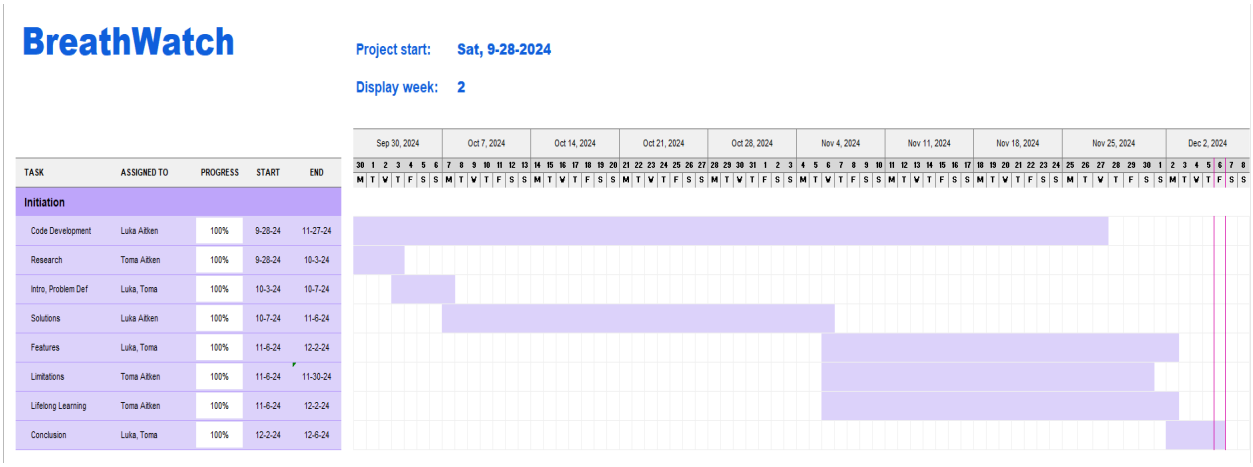


Figure 5.1 - Gantt Chart

6 Conclusion and Future Work

In conclusion, the BreathWatch system successfully meets its primary objective of monitoring patient health through real-time data collection and the application of design patterns to create an efficient, user-friendly interface. By leveraging the Singleton, Observer, Iterator, and Template design patterns, we developed a robust system that securely stores patient data, provides timely alerts for irregularities, and offers role-specific interfaces tailored for patients and clinicians. The system addresses key constraints such as usability, scalability, security, and accessibility, while ensuring a modular design that supports future enhancements. Additionally, the use of SQLite for local storage optimizes system performance and aligns with environmental and economic considerations by reducing reliance on energy-intensive cloud infrastructure.

Despite these achievements, there are limitations in the current prototype that provide avenues for future development. For instance, the lack of graphical visualizations such as charts and graphs limits clinicians' ability to analyze patient data trends at a glance. Furthermore, the notification system, while essential for real-time alerts, can overwhelm clinicians with multiple notifications when they log in. Addressing these challenges will enhance the overall user experience and system efficiency.

Future improvements to BreathWatch could include the addition of more user roles, such as administrators, with personalized profiles and histories to provide clinicians with a comprehensive view of patient data. A registration system for new patients would simplify the onboarding process and make the system accessible to a broader audience. Expanding the system to mobile platforms would improve accessibility and convenience for both patients and clinicians. Additionally, incorporating graphical visualizations such as trend graphs for vital signs would enhance data analysis and provide better insights. Finally, integrating advanced monitoring capabilities, such as wearable device support or multi-user monitoring sessions, could further expand the system's applicability in real-world healthcare settings.

Overall, the BreathWatch project demonstrates the potential of a well-designed health monitoring system to address critical healthcare needs while maintaining sustainability, safety, and accessibility. With future enhancements, BreathWatch has the potential to become a powerful tool for proactive and responsive healthcare management.

7 References

[1] M. Borrelli, "How to Breathe the Right Way," *Shondaland*, Jan. 13, 2022. [Online]. Available: https://www.shondaland.com/live/body/a38658028/how-to-breathe-the-right-way/. [Accessed: Oct 6, 2024].

[2] Cleveland Clinic, "Dyspnea (Shortness of Breath): Causes, Symptoms, and Treatment," Cleveland Clinic, [Online]. Available: https://my.clevelandclinic.org/health/symptoms/16942-dyspnea [Accessed: Oct 6, 2024].

[3] World Health Organization, “Respiratory Diseases: A Global Perspective,” 2021. [Online]. Available: [https://books.google.ca/books?hl=en&lr=&id=OaIOEQAAQBAJ&oi=fnd&pg=PR5&dq=World+Health+Organization.+(2021).+Respiratory+Diseases:+A+Global+Perspective.+&ots=p11R0spJ9m&sig=aU3-YIkOsAgcap-61qr3mlnGn74&redir_esc=y#v=onepage&q=World%20Health%20Organization.%20(2021).%20Respiratory%20Diseases%3A%20A%20Global%20Perspective.&f=false]. [Accessed: Oct 7,2024].

[4] S. Saha, et al., “Barriers to Accessing Healthcare for Low-Income Individuals,” Health Services Research. [Online]. Available: [https://www.tandfonline.com/doi/full/10.1080/07370016.2018.1404832#d1e111]. [Accessed: Oct 7, 2024].

[5] NetNada, "How to Reduce Carbon Emissions in Software Development," NetNada, [Online]. Available: https://www.netnada.com.au/post/how-to-reduce-carbon-emissions-in-software-development#:~:text=One%20of%20the%20most%20effective,algorithms%20and%20reducing%20unnecessary%20computations [Accessed: Nov 25, 2024].

8 Appendix

8.1 How to Set-up Application

Here are the steps you will need to install and run the BreathWatch application.

To download the code, you will need to go to GitHub and download the code. Link to GitHub can be found here: https://github.com/lukaaitken/BreathWatchApp.

After downloading the files, you will need to open BreathWatch in Intellij IDEA. After opening the source code in Intellij IDEA, you will need to install the plugin “SimpleSqliteBrowser”. This plugin will allow you to view the sql database from the sqlite library which should also be included in the source files

Once all the code and plugins are set up, head over to the BreathWatchApp file to initiate the program. When the program initiates, you will see the login screen. There are a total of 4 user accounts with passwords.

Username	Password
patient	123
patient2	456
patient3	789
clinician	123

Table 8.1.1 - User Accounts and Passwords

Each user’s username will be the user’s id. When you login as a patient account, the patientID will be the same as the username and can’t be changed by the user. From there, the patient can enter symptoms in a text field and enter values for breathing and heart rate or hit the “Generate Random Values” which will generate accurate numbers for breathing and heart rate. The patient view will also show you your average breathing and heart rate as well. Once the user is done, you can select the “Submit” button which will upload the data to the SQL database. Once submitted, you can exit out of application to close the program.

Signing into the clinician account is the same as the patient accounts. Only difference is what each account sees. Signing into the clinician will be able to see all submissions by patient accounts where they can also see the alerts for abnormal rates.